AGENTIC RAG: ROUTER-RETRIEVER SYSTEM

PDF and Web Search Tools

Course-End Project Write-up

Name: Shadli Islam

Implementation: Python 3.12.3, Jupyter Notebook, CrewAI

Knowledge source: “Attention Is All You Need” Transformer research paper

This report describes the architecture, implementation, demonstrations, results, challenges, and engineering trade-offs of a multi-agent retrieval-augmented generation system.

1. Project Overview

This project implements an Agentic Retrieval-Augmented Generation (RAG) system using CrewAI. The system uses two role-based AI agents to determine the appropriate source for a user question and produce an evidence-grounded response.

A **Router Agent** first classifies each question as **PDF** or **WEB**. Questions about the supplied Transformer research paper are routed to semantic PDF search. Questions that require recent, changing, or external information are routed to Tavily web search. A **Retriever Agent** then receives the retrieved evidence and creates a concise answer using only that evidence.

Workflow: User Question → Router Agent → PDF or Web Retrieval → Retriever Agent → Grounded Answer

2. Project Objectives

- Implement a Router Agent that selects the appropriate retrieval source.
- Implement a Retriever Agent that generates answers from retrieved evidence.
- Perform semantic retrieval from a supplied PDF using PDFSearchTool.
- Retrieve current external information using Tavily Search.
- Coordinate separate agent responsibilities using CrewAI.
- Generate source-grounded answers and avoid unsupported claims.
- Record operational trace information for each run.
- Demonstrate the full workflow using PDF and web questions.

3. System Architecture

Stage	Component	Responsibility
1	User Input	Accept a natural-language question.
2	Router Agent	Return exactly PDF or WEB based on information freshness and scope.
3A	PDFSearchTool	Search the supplied Transformer paper using local semantic embeddings.
3B	Tavily Search	Retrieve current web results for recent or external questions.
4	Retriever Agent	Answer using only the selected source evidence.
5	Operational Trace	Record question, route, tool, source, answer, timestamp, and duration.

4. Agent Responsibilities

Router Agent

The Router Agent analyzes the intent and information requirements of each question. It selects **PDF** for topics covered by the Transformer paper, including attention, multi-head attention, positional encoding, encoder-decoder architecture, training, and experimental results. It selects **WEB** for questions containing signals such as “latest,” “recent,” or “current,” or when external information is required.

Retriever Agent

The Retriever Agent receives both the original question and evidence from the selected source. The agent is instructed to answer only from the supplied evidence, clearly state when evidence is insufficient, avoid unsupported information, and end the response with the source type.

5. Tools and Technologies

Technology	Purpose
Python 3.12.3	Primary implementation language.
Jupyter Notebook	Development, testing, demonstration, and visible execution outputs.
CrewAI	Role-based multi-agent orchestration.
PDFSearchTool	Semantic retrieval from the Transformer PDF.
Sentence Transformers	Local generation of document and query embeddings.
all-MiniLM-L6-v2	Local embedding model used for PDF retrieval.
ChromaDB	Vector storage and similarity search.
Tavily Search	Retrieval of current web information.
Course Azure OpenAI gateway	Language model access for the Router and Retriever agents.
Custom CrewAI BaseLLM	Adapter for the course gateway's exact API endpoint.

6. Implementation Details

6.1 PDF Retrieval

The “Attention Is All You Need” Transformer paper was uploaded to the Jupyter working directory and used as the static RAG knowledge source. PDFSearchTool was configured with the local **all-MiniLM-L6-v2** sentence-transformer model and ChromaDB. This configuration was necessary because the course credentials did not include a separate Azure embedding deployment.

The PDF retrieval path was tested independently with the question “What is multi-head attention?” The tool returned relevant content from the paper before the retrieval output was connected to the Retriever Agent.

6.2 Web Retrieval

Tavily Search was configured for questions needing current or external information. The web path was tested independently before agent orchestration. A question asking for the latest developments in Transformer architectures produced web results that were then supplied to the Retriever Agent.

6.3 Custom Language Model Integration

The course provided access to an Azure OpenAI model through an Azure API Management gateway. CrewAI’s built-in Azure provider generated a URL structure that was incompatible with the supplied gateway, resulting in “Resource not found” responses.

A custom **CourseAzureLLM** class was therefore implemented by extending CrewAI’s **BaseLLM**. The adapter sends requests directly to the exact course endpoint using the required **api-key** header and returns the response content in the format expected by CrewAI. This custom adapter was validated independently before it was assigned to both agents.

6.4 Asynchronous Execution

Jupyter Notebook runs an active event loop. CrewAI’s synchronous **kickoff()** method conflicted with that environment. The crews were therefore executed with **await crew.akickoff()**, CrewAI’s native asynchronous execution pattern.

6.5 End-to-End Workflow

A reusable asynchronous function named **run_agentic_rag(question)** combines routing, retrieval, grounded generation, and trace logging. The function creates a routing task, validates the route, executes PDFSearchTool or Tavily, truncates excessively long evidence, invokes the Retriever Agent, and returns a structured trace dictionary.

7. Demonstration and Results

Test Question	Expected	Actual	Result
What is scaled dot-product attention?	PDF	PDF	PASS
Why does the Transformer use positional encoding?	PDF	PDF	PASS
What are the latest developments in Transformer architectures?	WEB	WEB	PASS

The demonstration shows that the Router Agent distinguished static, domain-specific questions from a current-information question. The PDF questions used PDFSearchTool, while the current-information question used Tavily Search. The Retriever Agent produced grounded responses and identified the source used for each answer.

8. Logging and Traceability

Each execution returns and prints an operational trace containing:

- Timestamp
- User question
- Router decision
- Retrieval tool
- Information source
- Final answer
- Execution duration in seconds

These fields make the workflow observable without exposing private chain-of-thought. The trace shows what the system received, which route was selected, which tool was executed, and what answer was produced.

9. Challenges Encountered and Resolutions

Externally managed Python environment

The VM prevented global pip installation. A Python 3.12 virtual environment was created and registered for Jupyter.

Kernel and interpreter mismatch

The notebook initially used /usr/bin/python3. Jupyter was launched from the clean virtual environment so the notebook used the project interpreter.

Missing embedding deployment

The course Azure credentials provided a chat model but no embedding deployment. Local all-MiniLM-L6-v2 embeddings were used.

Default PDFSearchTool configuration

PDFSearchTool initially expected a standard OpenAI API key. The tool was explicitly configured with sentence-transformer embeddings and ChromaDB.

Azure gateway incompatibility

CrewAI's native Azure provider generated a gateway path that returned 404 errors. A custom BaseLLM adapter used the exact course endpoint.

Jupyter event-loop conflict

Synchronous crew execution raised a running-event-loop error. The workflow was changed to await crew.akickoff().

10. Engineering Trade-offs

- **Local embeddings:** avoid another paid API and keep PDF indexing self-contained, but retrieval quality may differ from larger commercial embedding models.
- **Static PDF retrieval:** provides controlled and reproducible evidence, but it cannot reflect information published after the paper.
- **Web retrieval:** supplies current information, but the answer quality depends on search result relevance and source quality.
- **Separate agents:** improve modularity, role clarity, and traceability, but add model calls and execution latency.
- **Custom LLM adapter:** enables compatibility with the course gateway, but adds code that must be maintained if the course endpoint contract changes.

11. Security and Submission Practices

API credentials were entered at runtime with hidden input rather than written directly into the notebook. The final notebook and screenshots should not contain the Azure API key, Tavily API key, or the course credential file. Failed traceback cells and credential-testing experiments should be removed before export.

12. Conclusion

The project successfully demonstrates an Agentic RAG architecture capable of routing questions between a static PDF knowledge source and current web information. The Router Agent selects the appropriate retrieval method, PDFSearchTool or Tavily retrieves evidence, and the Retriever Agent creates a grounded answer from that evidence.

The final implementation demonstrates multi-agent coordination, semantic PDF retrieval, web retrieval, source-grounded generation, asynchronous orchestration, and operational traceability. The real integration challenges were resolved through a Python virtual environment, local sentence-transformer embeddings, a custom CrewAI BaseLLM adapter, and native asynchronous crew execution.

References

1. Vaswani, A., et al. “Attention Is All You Need.” 2017.
2. CrewAI documentation: PDFSearchTool, custom LLM implementation, and asynchronous crew execution.
3. Tavily documentation: Search API and API credits.
4. Course-End Project problem statement: Agentic RAG Router-Retriever System with PDF and Web Search Tools.